

Instruções Manuais ou Automáticas? Avaliando Embeddings Instrucionais para Detecção de Linguagem Ofensiva em Português Brasileiro

Mariana da Silva Martins¹, Andre Maxwell Vasconcelos Barbosa Junior¹, Lucas Nóbrega Albuquerque¹, Francelino Teotonio Júnior¹, Yuri de Almeida Malheiros²

¹Centro de Informática – Universidade Federal da Paraíba (UFPB)
João Pessoa – PB – Brasil

²Universidade Federal de Pernambuco (UFPE) – PE – Brasil.

{marianamartiyms, contato.andremaxwell, francelinojunior_,
lucasnobalb}@gmail.com, yuri@ci.ufpb.br

Resumo. Este trabalho investiga se instruções geradas automaticamente por uma LLM são uma alternativa viável a instruções manuais em embeddings instrucionais para detecção de linguagem ofensiva em português brasileiro. Usando o dataset HateBR, comparamos três modelos de embeddings (e5_multilingual, e5_large_instruct e bertimbau_pt, este último como controle não instrucional) sob três condições (sem instrução, instrução manual fixa e instrução automática por instância), treinando um MLP idêntico para cada combinação. Para os modelos com ajuste instrucional mais raso e para o controle, a diferença entre instrução manual e automática não é estatisticamente significativa; já no modelo com treinamento instrucional mais extenso, a manual supera a automática de forma significativa e robusta a variações de semente. A ausência de instrução igualou ou superou as condições instruídas, e todas as combinações superaram os baselines de TF-IDF e heurística lexical.

Palavras-chave: Discurso de ódio. Embeddings instrucionais. Geração automática de instruções. Processamento de Linguagem Natural. Modelos de linguagem.

1. Introdução

A moderação de conteúdo online é um elemento central para tornar ambientes digitais mais seguros. Em redes sociais, fóruns e demais plataformas, usuários publicam com frequência mensagens ofensivas, discriminatórias ou agressivas, o que afeta diretamente a participação de pessoas e grupos vulneráveis nesses espaços. No campo do Processamento de Linguagem Natural, esse problema costuma ser tratado como uma tarefa de classificação de texto: dado um comentário, o modelo decide se ele é ofensivo ou não. A dificuldade central é que esse tipo de linguagem nem sempre aparece de forma explícita, o uso de ironia, gírias, indiretas e ataques disfarçados são comuns, o que torna a representação do texto uma etapa crítica do problema.

Embeddings são representações vetoriais de texto amplamente utilizadas como entrada para modelos de aprendizado de máquina. Modelos de embedding instrucional,

como o INSTRUCTOR (Su et al., 2022), avançam esse paradigma ao permitir que uma instrução em linguagem natural seja fornecida junto ao texto, orientando o modelo a destacar aspectos específicos relevantes para a tarefa em questão. Entretanto, usuários frequentemente expressam suas necessidades por meio de consultas vagas, imprecisas ou incompletas, dificultando a interpretação adequada de suas intenções pelos sistemas baseados em LLMs (Large Language Models, ou Modelos de Linguagem de Grande Escala) e criando uma lacuna entre a intenção do usuário e o comportamento esperado do modelo (Yuan et al., 2025).

A partir dessa lacuna, a pergunta central deste trabalho é: *instruções geradas automaticamente por LLMs são comparáveis a instruções escritas manualmente para embeddings instrucionais na detecção de linguagem ofensiva em português brasileiro?* Para respondê-la, comparamos três condições: (i) o modelo recebe apenas o comentário, sem instrução, como referência de baseline; (ii) o modelo recebe o comentário junto a uma instrução fixa, escrita manualmente com base nas categorias de discurso de ódio presentes no dataset; e (iii) o modelo recebe o comentário junto a uma instrução gerada automaticamente por uma LLM a partir do próprio texto do comentário.

A contribuição deste trabalho é avaliar empiricamente se a geração automática de instruções via LLM, em tempo de inferência (ou seja, no momento da predição, sem necessidade de uma etapa prévia de treinamento ou busca de instruções), constitui uma alternativa viável à instrução manual, reduzindo a dependência de especialistas na elaboração de boas instruções e tornando o uso de embeddings instrucionais mais acessível na prática. Os experimentos são conduzidos sobre o dataset HateBR (VARGAS; CARVALHO, 2025), um corpus consolidado de comentários em português brasileiro anotados quanto à ofensividade, incluindo discurso de ódio entre os fenômenos capturados pela anotação.

O restante deste artigo está organizado da seguinte forma: a Seção 2 discute os trabalhos relacionados e posiciona nossa contribuição frente à literatura; a Seção 3 descreve a metodologia, incluindo o dataset, o modelo de *embeddings*, as estratégias de geração de instrução e o protocolo experimental; em seguida as Seções 4 e 5 com Resultados e Conclusão.

2. Trabalhos Relacionados

2.1 Embeddings instrucionais

A idéia de condicionar embeddings de texto a instruções em linguagem natural foi consolidada por Su et al. (2022) com o INSTRUCTOR, um modelo que recebe, além do texto, uma instrução descrevendo a tarefa e o domínio, permitindo que o mesmo texto gere

representações distintas conforme o objetivo final. Treinado sobre o MEDI, uma coleção de 330 datasets com instruções, o INSTRUCTOR superou modelos até dez vezes maiores em benchmarks de classificação, clustering e similaridade semântica, demonstrando que o ganho vem do condicionamento por instrução e não apenas da escala do modelo. Asai et al. (2022), com o TART, aplicaram o mesmo paradigma à recuperação de informação, mostrando que instruções de tarefa (intenção, domínio e unidade do resultado esperado) permitem que um único retriever se adapte a tarefas nunca vistas sem re-treinamento. Wang et al. (2024), com a família mE5, estenderam o paradigma a cenários multilíngues, mostrando que o fine-tuning por instrução (mE5-large-instruct) melhora a generalização entre tarefas mesmo quando não supera uniformemente cada benchmark isolado.

Em comum, esses três trabalhos tratam a instrução como fixa por tarefa: uma única instrução, escrita manualmente uma vez, é reaproveitada para todos os exemplos daquela tarefa. Essa é precisamente a limitação que motiva o presente trabalho, a dependência de um especialista para escrever, a priori, uma boa instrução por tarefa.

2.2 Geração automática de instruções

Paralelamente, uma linha de pesquisa buscou automatizar a própria criação de instruções, reduzindo a dependência de prompt engineering manual. Zhou et al. (2022) propuseram o APE (Automatic Prompt Engineer), que trata a instrução como um "programa" otimizado por busca: um LLM gera instruções candidatas a partir de poucos exemplos de entrada e saída, e cada candidata é avaliada quanto ao desempenho real na tarefa, sendo a melhor selecionada. O método igualou ou superou instruções humanas em 24 das 24 tarefas de Instruction Induction. Zhang et al. (2023), com o Auto-Instruct, avançaram essa ideia para o cenário *true few-shot*, no qual não há dados de validação disponíveis: um modelo de ranking (FLAN-T5) treinado sobre centenas de tarefas aprende a estimar, sem executar cada candidata no LLM-alvo, qual instrução tende a funcionar melhor para um exemplo específico, inclusive selecionando instruções diferentes para exemplos diferentes da mesma tarefa.

Ambos os trabalhos otimizam a instrução offline e por tarefa, exigindo uma etapa prévia de busca, geração de candidatas e avaliação. O presente trabalho se diferencia por gerar a instrução online e por instância, diretamente a partir do conteúdo do comentário a ser classificado, sem etapa de otimização ou conjunto de validação, o que reduz o custo computacional e dispensa a curadoria manual de instruções por especialistas, ao custo de não haver uma etapa explícita de seleção entre candidatas.

2.3 Detecção de discurso de ódio em português com LLMs

Diversos trabalhos recentes investigam o uso de LLMs para classificação de discurso tóxico em português brasileiro, majoritariamente com o dataset ToLD-Br como benchmark. Oliveira et al. (2023) avaliam o ChatGPT-3.5 Turbo em modo zero-shot para detecção de hate speech em tweets, comparando-o com modelos encoder ajustados como BERTimbau-large. Os resultados mostram que o ChatGPT com um prompt de escopo amplo atinge F1-macro de 0,73 no ToLD-Br, contra 0,75 do BERTimbau ajustado, mas supera esse modelo em avaliação cruzada sobre um dataset europeu, evidenciando maior robustez a mudanças de distribuição. Oliveira et al. (2024a) ampliam essa comparação, incluindo o MariTalk, especializado em português, e investigam o impacto do few-shot learning. Os autores identificam que o MariTalk com 10 exemplos atinge desempenho competitivo com o ChatGPT (F1 de 0,73) e demonstra maior sensibilidade à linguagem coloquial brasileira, classificando corretamente palavrões usados como intensificadores em vez de ofensas genuínas. Ambos os estudos apontam que nenhum LLM superou o BERTimbau ajustado, mas que modelos menores e monolíngues podem ser alternativas viáveis.

Ghorbanpour, Dementieva e Fraser (2025) investigam o papel do design de prompts de forma mais sistemática, avaliando dezenas de estratégias de prompting, de vanilla e chain-of-thought a role-play, NLI e few-shot com até cinco exemplos, em oito idiomas não-ingleses, com modelos como LLaMA 3.1 8B e Qwen2.5 7B. Os autores mostram que não existe uma estratégia de prompt universalmente superior, cada idioma tende a responder melhor a uma técnica específica, e que o few-shot com cinco exemplos foi a configuração mais eficaz na maioria dos casos. Esse resultado reforça a motivação do presente trabalho: se o design da instrução é crítico para o desempenho, estratégias que reduzam a dependência de especialistas na elaboração dessas instruções tornam-se especialmente relevantes.

Nessa mesma direção, Oliveira et al. (2024b) propõem um método de evolução iterativa de prompts inspirado em algoritmos genéticos para otimizar automaticamente as instruções passadas ao LLaMA 3.1 8B na tarefa de classificação de toxicidade em português. O processo utiliza o GPT-4o mini como operador de mutação e recombinação, refinando prompts ao longo de 50 épocas com base no F1-score obtido. O melhor prompt identificado é então utilizado para fine-tuning via QLoRA, elevando o F1 do modelo de 0,61 para 0,75. Embora o foco deste trabalho seja a otimização de prompts para fine-tuning, a abordagem compartilhar com o presente estudo a premissa de que a geração automática de instruções pode substituir ou superar a engenharia manual, e os resultados demonstram que a qualidade das instruções geradas impacta diretamente o desempenho final.

2.4 Classificação de intenção e estruturação linguística

Zhang e Bertaglia (2025) propõem o LinGO, um framework voltado à interpretação estruturada da intenção em discursos incivis, distinguindo padrões como ataques explícitos, ataques implícitos, contra-discurso e escalada. Diferentemente do presente trabalho, o objetivo do LinGO está na decomposição linguística do processo de inferência e na identificação da intenção comunicativa. Neste estudo, por outro lado, mantém-se uma tarefa de classificação binária, investigando-se se instruções associadas ao texto podem modificar sua representação vetorial de maneira favorável à classificação. Assim, embora os objetivos sejam distintos, o LinGO reforça uma motivação relevante para este trabalho: elementos como ironia, ataques indiretos e conteúdo implícito podem exigir uma interpretação além da superfície lexical, tornando pertinente investigar estratégias capazes de orientar a representação semântica dos comentários. A Tabela 1 sintetiza essa comparação entre os trabalhos relacionados.

Tabela 1 – Comparação das abordagens quanto ao nível e à origem da instrução

Trabalho	Tipo de abordagem	Nível da instrução	Origem/momento da instrução	Diferença para este projeto
Su et al. (2022): INSTRUCTOR	Embeddings instrucionais	Por tarefa (fixa)	Manual, curada por especialistas	Define o paradigma de embeddings condicionados por instrução, mas exige instrução escrita a priori para cada tarefa
Asai et al. (2022): TART	Retrieval instrucional	Por tarefa (fixa)	Manual	Aplica o mesmo paradigma à recuperação de informação; instrução continua fixa e escrita manualmente
Wang et al. (2024): mE5	Embeddings instrucionais multilíngues	Por tarefa (fixa)	Manual + sintética, gerada uma vez, offline	Usa LLM para gerar instruções sintéticas, mas ainda por tarefa e antes da inferência, não a partir do comentário específico
Zhou et al. (2022): APE	Geração automática de prompts	Por tarefa	Automática, via busca e seleção offline com dados de validação	Otimiza uma única instrução por tarefa; não gera por instância nem em tempo de inferência

Zhang et al. (2023): Auto-Instruct	Geração/seleção automática de prompts	Por instância (seleção entre candidatas pré-geradas)	Automática, ranking treinado, aplicado exemplo a exemplo	Seleciona entre candidatas geradas previamente para a tarefa; não gera uma instrução nova a partir do conteúdo do exemplo
Este projeto	Embeddings instrucionais + geração automática	Por instância	Automática, gerada em tempo de inferência a partir do próprio comentário	-

Fonte: Elaborado pelos autores (2026), com base em Su et al. (2022), Asai et al. (2022), Wang et al. (2024), Zhou et al. (2022) e Zhang et al. (2023).

Os trabalhos revisados evidenciam dois movimentos paralelos na literatura: de um lado, a avaliação de LLMs como classificadores de discurso de ódio em português, com atenção crescente ao design de prompts como fator determinante de desempenho; de outro, iniciativas de automação dessas instruções, seja por otimização iterativa, seja por estruturação linguística da inferência. Enquanto os trabalhos sobre embeddings instrucionais (Su et al., 2022; Asai et al., 2022; Wang et al., 2024) fixam uma instrução por tarefa, e os trabalhos de geração automática de instruções (Zhou et al., 2022; Zhang et al., 2023) otimizam essa instrução offline e por tarefa, nenhum deles investiga a geração de instruções por instância, em tempo de inferência, como entrada para um modelo de embeddings instrucionais aplicado à detecção de linguagem ofensiva, a combinação específica que este trabalho avalia no contexto do português brasileiro. Enquanto os trabalhos de Oliveira et al. (2024a) e Ghorbanpour et al. (2025) operam no nível do prompt de LLMs generativas e avaliam a saída direta do modelo, o presente trabalho se situa em um ponto distinto da cadeia: usa a LLM apenas para enriquecer a representação do texto via instrução, mantendo a decisão de classificação em um modelo leve treinado sobre os embeddings. A Tabela 2 resume os trabalhos relacionados discutidos nesta seção.

Tabela 2 – Trabalhos relacionados à detecção de discurso de ódio e toxicidade em português.

Trabalho	Tarefa	Dados	Técnica	Diferença para este projeto
Oliveira et al. (2023)	Detecção de Hate Speech	ToLD-Br	ChatGPT (Zero-shot)	Foca na saída direta da LLM como classificador; este projeto usa a LLM só para gerar instruções para embeddings

Ghorbanpour et al. (2025)	Detecção Multilíngue	Diversos	Prompt Engineering (Few-shot)	Explora manualmente estratégias de prompt por idioma; este projeto automatiza a instrução por instância, sem busca de estratégia
Oliveira et al. (2024a)	Classificação de Toxicidade	ToLD-Br	Evolução de Prompts (Genético)	Otimiza um prompt por tarefa, offline, para fine-tuning; este projeto gera instruções dinâmicas por comentário, em tempo de inferência
Zhang & Bertaglia (2025)	Intenção de Incivilidade	-	LinGO (Grafos Linguísticos)	Foca na interpretação estruturada da intenção; este projeto foca na qualidade da representação (embedding) obtida via instrução
Este Projeto	Detecção de Linguagem Ofensiva	HateBR	Embeddings Instrucionais + LLM AutoPrompt	

Fonte: Elaborada pelos autores (2026), com base em Oliveira et al. (2023), Ghorbanpour, Dementieva e Fraser (2025), Oliveira et al. (2024a) e Zhang e Bertaglia (2025).

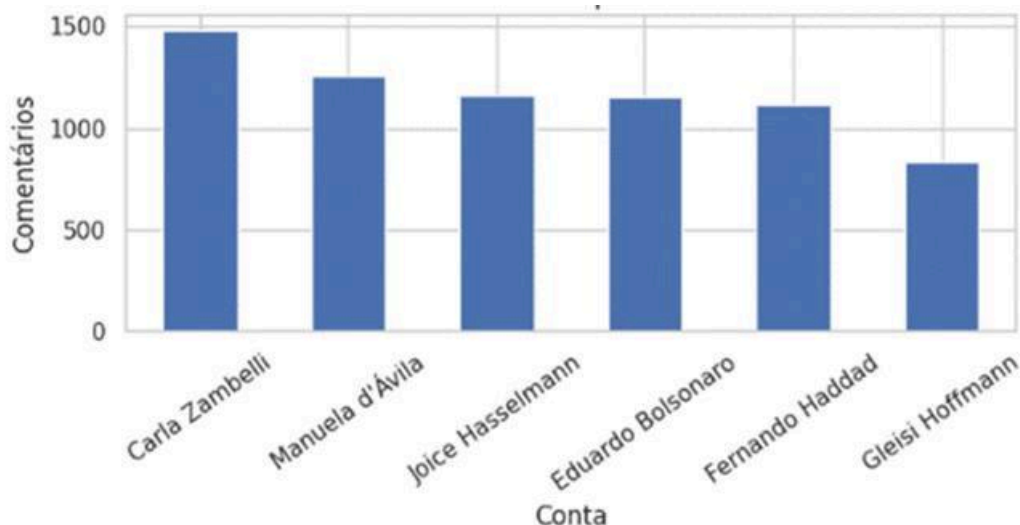
Diferente dos trabalhos que utilizam LLMs como classificadores finais (black-box), nossa abordagem utiliza a LLM apenas para enriquecer a representação do texto via instrução. Isso permite manter o classificador final leve e interpretável, e isolando o impacto da qualidade da instrução na formação do embedding. A lacuna preenchida reside na avaliação dessa automação em tempo de inferência para o português brasileiro.

3. Metodologia

3.1 Base de Dados

A condução das etapas experimentais utilizou o HateBR, um corpus consolidado para o português brasileiro voltado à identificação de linguagem ofensiva e discurso de ódio em mídias sociais. O dataset foi elaborado a partir de comentários coletados no Instagram, especificamente em perfis de figuras políticas nacionais, como ilustra a Figura 1, contando com a curadoria de especialistas para a anotação manual. Cada instância textual passou pela análise independente de três juízes, garantindo uma robusta concordância interna aos anotadores e elevando o grau de confiabilidade das etiquetas presentes no corpus (VARGAS; CARVALHO, 2025).

Figura 1 – comentários por conta



Estruturalmente, o HateBR reúne 7.000 entradas e oferece múltiplas perspectivas de rotulagem: (i) detecção binária de ofensividade, (ii) gradação da toxicidade (leve, moderada ou severa) e (iii) mapeamento do alvo do ataque. Para este estudo, focou-se exclusivamente na classificação binária, que possui um equilíbrio ideal entre as classes (3.500 exemplos para não ofensivo e 3.500 para ofensivo), premissa essencial para mensurar o impacto das instruções na geração de representações vetoriais instrucionais. A Tabela 3 apresenta exemplos representativos de comentários de cada classe.

Tabela 3 – Exemplos de comentários por classe

Ofensivo (1)	Não ofensivo (0)
“O tombo vai ser grande, eu vou aplaudir de pé.”	“Também acho. Esse cara fala demais. Os filhos do bolsonaro acabam enfraquecendo o governo. Sempre disse isso desde o começo”
“Até quando o STF vai trabalhar contra o país?”	“Benedita não se altere porque esse ato cometido foi por gente que tem alma escura !”
“O grupo é grande, com certeza tem mais políticos no meio.”	“Lei da mordaca”

A partição dos dados seguiu uma proporção de 70% (4.900), 15% (1.050) e 15% (1.050) para os conjuntos de treino, validação e teste, respectivamente. O processo foi executado de maneira estratificada pelo rótulo principal, assegurando que a distribuição original das classes fosse mantida em todos os subconjuntos criados.

No pré-processamento, definiu-se a exclusão de registros com lacunas de texto ou de anotação, bem como entradas nulas após a limpeza de espaços; contudo, a integridade da base foi mantida com os 7.000 comentários originais, visto que nenhum item violou esses

critérios. Abstemo-nos de empregar normatizações como lematização ou remoção de stopwords, pois os modelos de *embeddings* instrucionais operam de forma otimizada sobre o fluxo natural da linguagem. Tal decisão visou blindar a riqueza semântica e os matizes pragmáticos do corpus, preservando gírias e coloquialismos inerentes à expressão do discurso de ódio no cenário digital brasileiro.

3.2 Estratégias de Geração de Instruções

Neste estudo, as representações vetoriais são extraídas sob três paradigmas distintos: (i) *baseline* sem instrução, onde apenas o texto bruto do comentário é processado; (ii) instrução manual fixa, composta por uma diretriz estática elaborada pelos autores e aplicada uniformemente a todo o corpus; e (iii) instrução automática por instância, na qual uma LLM gera uma orientação customizada e concisa para cada comentário, visando capturar nuances semânticas fundamentais para a identificação de discurso de ódio naquele contexto específico.

A diretriz manual adotada para os modelos da família E5 foi redigida em inglês: “*Represent the following Brazilian Portuguese comment for hate speech detection. Pay attention to insults, discrimination, prejudice, attacks against minorities and offensive language.*”. Para o modelo BERTimbau, utilizou-se a respectiva tradução para o português: “*Represente o seguinte comentário em português brasileiro para detecção de discurso de ódio. Preste atenção a insultos, discriminação, preconceito, ataques a minorias e linguagem ofensiva.*”

Para os modelos E5, a entrada estruturada segue o padrão nativo *Instruct*: *<instrução> e Query: <texto>*. No BERTimbau, que atua como controle não-instrucional, a instrução é simplesmente prefixada ao comentário. A vertente automatizada utiliza o modelo llama-3.1-8b-instant via API do Groq, com um *prompt* que solicita uma instrução única de no máximo 25 palavras, estritamente vinculada ao conteúdo do exemplo. As requisições são processadas em lotes de dez comentários com concorrência quádrupla. Em cenários de falha no lote, o sistema reprocessa cada item individualmente. Visando a eficiência experimental, todas as instruções e seus respectivos *embeddings* são persistidos em cache local.

A avaliação abrange três arquiteturas disponibilizadas pela biblioteca *sentence-transformers*:

- `intfloat/multilingual-e5-base` (`e5_multilingual`): Modelo base multilíngue que, embora utilize a formatação instrucional, não passou por fine-tuning específico para seguimento de instruções livres, servindo como controle de contexto;

- intfloat/multilingual-e5-large-instruct (e5_large_instruct): Variante robusta com treinamento instrucional extensivo, sendo o foco central da análise;
- rufimelo/bert-large-portuguese-cased-sts (bertimbau_pt): Codificador especializado em português brasileiro, desprovido de ajuste instrucional, utilizado como referência comparativa de baseline.

O BERTimbau não é empregado como modelo instrucional, mas como condição de controle, permitindo distinguir possíveis ganhos associados à capacidade de seguir instruções daqueles decorrentes apenas da adição de contexto textual à entrada. Para assegurar que as variações de desempenho derivem exclusivamente da qualidade da representação vetorial, um Perceptron Multicamadas (MLP) com arquitetura idêntica é treinado sobre os *embeddings* para cada combinação de modelo e estratégia de instrução.

3.3 Configurações de Referência e Baselines

A condição sem instrução atua como o referencial interno de cada arquitetura. Os modelos e5_multilingual e bertimbau_pt permitem isolar se eventuais ganhos decorrem da capacidade instrucional ou meramente da adição de contexto textual. Além destes, dois *baselines* externos são empregados:

- TF-IDF + Regressão Logística: Modelo baseado em n-gramas com vocabulário de 20.000 termos;
- Heurística Lexical: Regra de decisão fundamentada em um léxico pejorativo de 27 termos frequentes no português brasileiro (ex: “verme”, “canalha”).

O protocolo experimental abrange nove combinações principais resultantes do cruzamento entre os três modelos de *embeddings* e as três modalidades de instrução, além dos marcos comparativos externos.

3.4 Protocolo de Avaliação

A performance é quantificada via acurácia, precisão, revocação e *F1-score*. Devido ao equilíbrio das classes no HateBR, o F1-macro é estabelecido como métrica primária. Complementarmente, análises via matrizes de confusão permitem o escrutínio dos tipos de erro predominantes em cada configuração. Para além da avaliação pontual em um único treinamento, o protocolo inclui três verificações complementares: (i) uma checagem de robustez, retreinando o MLP em cinco sementes aleatórias distintas por combinação de modelo e condição; (ii) o teste pareado de McNemar, aplicado as previsões de teste para comparar estatisticamente instrução manual e automática em cada modelo; e (iii) uma análise geométrica do espaço de embeddings, via projeção por PCA e similaridade de cosseno intra e interclasse, para verificar se as instruções alteram a separação entre as classes no espaço vetorial.

3.5 Aspectos de Implementação

O arcabouço tecnológico foi desenvolvido em Python 3, utilizando as bibliotecas *sentence-transformers* para geração de vetores e *scikit-learn* para modelagem e métricas. Os hiperparâmetros críticos incluem: SEED = 42; arquitetura MLP com camadas (256, 128) e ativação ReLU; temperatura de 0,2 para a LLM; e normalização L2 nos *embeddings*.

A persistência de dados utiliza arquivos JSON para diretrizes e NPY para vetores. Os resultados consolidados são exportados em formato CSV para posterior análise estatística. Os modelos de linguagem empregados são de acesso público, exigindo-se apenas credenciais de API próprias para a replicação da etapa de geração automática.

4. Resultados e Discussão

4.1 Desempenho Geral entre Modelos e Condições de Instrução

A Tabela 4 sintetiza o F1-macro obtido no conjunto de teste para as nove combinações resultantes do cruzamento entre os três modelos de embeddings (e5_multilingual, e5_large_instruct e bertimbau_pt) e as três condições de instrução (sem instrução, instrução manual e instrução automática). A Figura 2 apresenta os mesmos valores em formato gráfico, sobrepostos às duas referências de baseline externo.

Tabela 4 - F1-macro no conjunto de teste, por modelo de embeddings e condição de instrução

Modelo	Sem instrução	Instrução manual	Instrução automática
e5_multilingual	0,879	0,864	0,870
e5_large_instruct	0,902	0,899	0,871
bertimbau_pt (controle)	0,882	0,893	0,888

Fonte: Elaborada pelos autores (2026), a partir dos resultados do notebook experimental.

O padrão mais evidente na Tabela 4 é que, para os dois modelos efetivamente treinados para seguir instruções, a condição sem instrução obteve o melhor ou o segundo melhor desempenho: 0,902 para e5_large_instruct (superando a instrução manual em cerca de 0,3 ponto percentual e a automática em quase 3 pontos percentuais) e 0,879 para e5_multilingual (superando ambas as condições instruídas). Esse resultado é, à primeira vista, contraintuitivo frente à premissa de que embeddings instrucionais deveriam se beneficiar de uma instrução explícita voltada à tarefa. Uma leitura possível é que, diferentemente do cenário de recuperação zero-shot em que o INSTRUCTOR e a família E5 foram originalmente validados, aqui um classificador MLP é treinado especificamente sobre cada conjunto de embeddings; esse classificador pode aprender a compensar a ausência de instrução explorando diretamente a geometria do espaço vetorial não instruído, reduzindo a vantagem relativa de instruções textuais adicionadas à entrada.

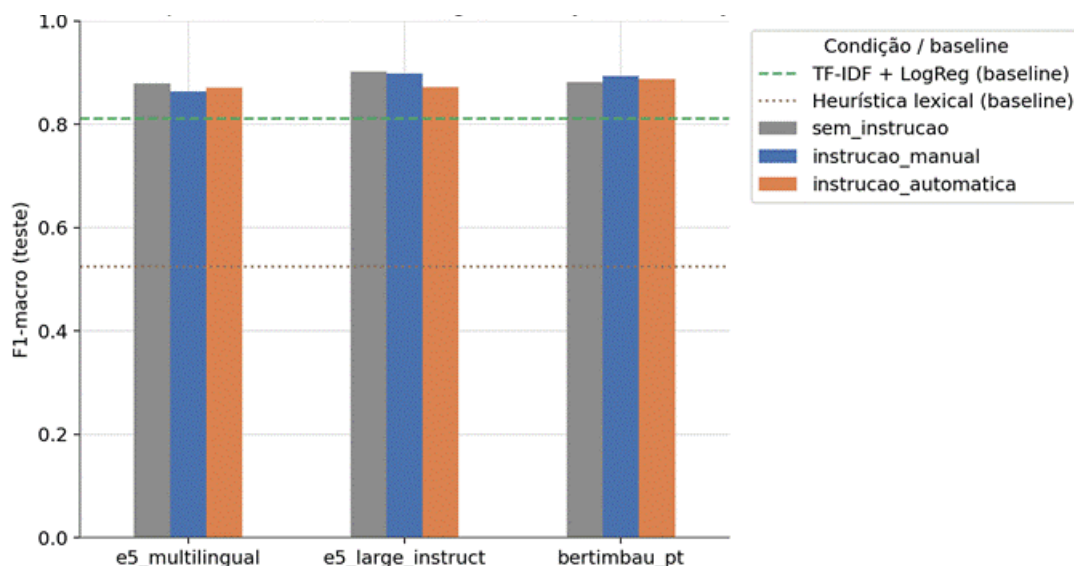
Dentro do recorte que constitui a pergunta central deste trabalho, instrução manual versus instrução automática, o comportamento difere por modelo. Para `e5_multilingual`, a instrução automática supera discretamente a manual (0,870 contra 0,864). Para `e5_large_instruct`, ocorre o inverso e com uma diferença mais expressiva: a instrução manual supera a automática por quase 3 pontos percentuais (0,899 contra 0,871). Já para o `bertimbau_pt`, que atua apenas como controle não instrucional, a instrução manual também obteve o melhor resultado entre as três condições (0,893), seguida da automática (0,888) e da ausência de qualquer texto adicional (0,882), um efeito que, nesse caso, deve ser lido como o impacto de concatenar contexto textual à entrada, e não como evidência de que o modelo "segue" a instrução, já que o BERTimbau não foi treinado para esse fim. As seções 4.3 e 4.4 avaliam, respectivamente, a robustez e a significância estatística dessas diferenças.

4.2 Comparação com os Baselines Externos

Os dois marcos comparativos externos definidos na Seção 3.3 situam o desempenho dos classificadores baseados em embeddings em perspectiva. A Heurística Lexical, que apenas verifica a presença de termos de um léxico pejorativo fixo, obteve F1-macro de 0,524 no teste, próximo do desempenho de um classificador aleatório equilibrado, evidenciando que grande parte do discurso de ódio no HateBR não se manifesta por meio de palavrões ou termos obviamente ofensivos, mas por construções mais sutis, irônicas ou implícitas. Já o TF-IDF combinado com Regressão Logística alcançou 0,810, um baseline competitivo baseado em frequência de n-gramas.

Como mostra a Figura 2, todas as nove combinações de modelo de embeddings e condição de instrução superam confortavelmente ambos os baselines externos, com uma margem mínima de cerca de 5 pontos percentuais de F1-macro sobre o TF-IDF + Regressão Logística. Esse resultado confirma que representações vetoriais densas, sejam elas instruídas ou não, capturam sinais mais relevantes para a detecção de discurso de ódio em português do que abordagens baseadas puramente em frequência lexical. Ou seja: independentemente da disputa entre instrução manual e automática, a escolha de uma arquitetura de embeddings pré-treinada já representa, isoladamente, o ganho mais expressivo de desempenho observado no experimento.

Figura 2 - F1-macro por modelo de embeddings e condição de instruções, com referência aos baselines externos



Fonte: Elaborada pelos autores (2026).

4.3 Robustez à Variação de Seed do Classificador

Para verificar se as diferenças reportadas na Seção 4.1 refletem um padrão consistente, e não uma particularidade da inicialização aleatória do MLP, cada uma das nove combinações foi retreinada em cinco sementes distintas (42, 7, 123, 2024 e 99), mantendo fixos os embeddings já calculados. A Tabela 5 e a Figura 3 reportam a média e o desvio padrão do F1-macro no teste ao longo dessas cinco execuções.

Tabela 5 - F1-macro no teste (média +/- desvio padrão em 5 sementes), por modelo e condição

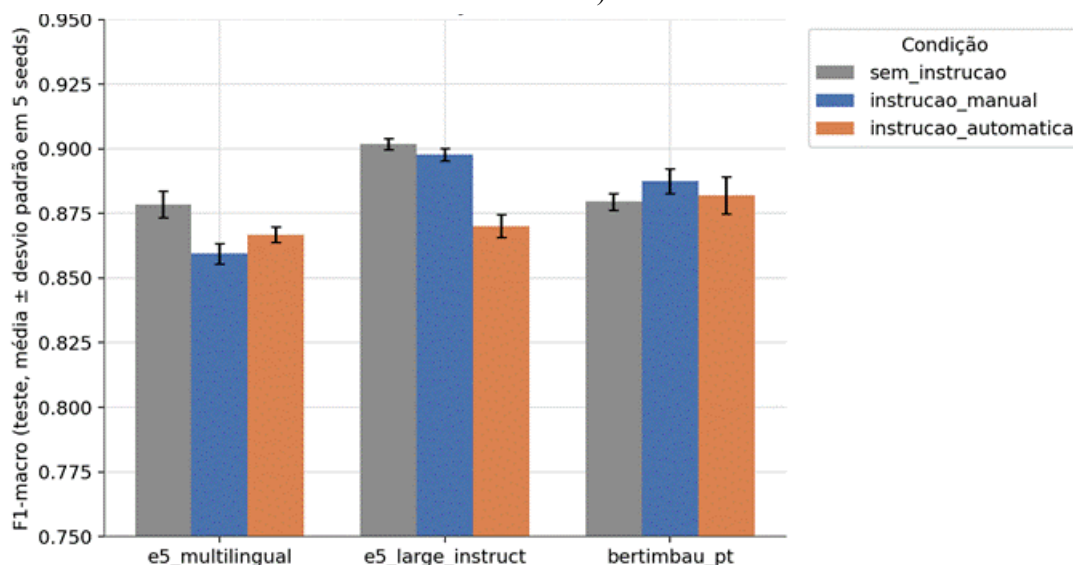
Modelo	Sem instrucao	Instrução manual	Instrução automática
e5_multilingual	0,878 +/- 0,005	0,859 +/- 0,004	0,867 +/- 0,003
e5_large_instruct	0,902 +/- 0,002	0,898 +/- 0,002	0,870 +/- 0,004
bertimbau_pt (controle)	0,880 +/- 0,003	0,888 +/- 0,005	0,882 +/- 0,007

Fonte: Elaborada pelos autores (2026)

Os desvios padrão observados são pequenos em todas as combinações (entre 0,002 e 0,007 de F1-macro), e as médias em cinco sementes reproduzem de perto os valores obtidos com a semente única utilizada na Seção 4.1. Isso indica que a diferença de quase 3 pontos percentuais entre instrução manual (0,898) e instrução automática (0,870) observada para o e5_large_instruct não é explicada pela variância do treinamento do classificador: os intervalos definidos pela média $\pm$ desvio padrão das duas condições não se sobrepõem. O mesmo não ocorre para e5_multilingual e para o bertimbau_pt, cujas diferenças entre instrução manual e automática são da mesma ordem de grandeza que o desvio padrão entre sementes, sugerindo maior cautela antes de interpretar essas diferenças menores como um

efeito real da instrução. A Seção 4.4 formaliza essa distinção por meio de um teste de significância estatística.

Figura 3 - Robustez do F1-macro a variação de seed do classificador (média +/- desvio padrão, 5 sementes)



Fonte: Elaborada pelos autores (2026).

4.4 Significância Estatística entre Instrução Manual e Automática

Para testar formalmente se a diferença de acertos entre a instrução manual e automática é estatisticamente significativa, aplicou-se o teste pareado de McNemar às previsões de teste de cada modelo instrucional, comparando os mesmos 1.050 comentários nas duas condições. A Tabela 6 apresenta a estatística do teste e o p-valor correspondente para os três modelos avaliados.

Tabela 6 - Teste de McNemar entre instrução manual e instruções automática, por modelo

Modelo	Estatística	p-valor	Significativo (alfa = 0,05)
e5_multilingual	0,275	0,600	Não
e5_large_instruct	7,327	0,007	Sim
bertimbau_pt (controle)	0,255	0,614	Não

Fonte: Elaborada pelos autores (2026)

O teste confirma a leitura da Seção 4.3: apenas para o e5_large_instruct a diferença entre instrução manual e automática é estatisticamente significativa ($p = 0,007$), com a instrução manual produzindo mais acertos discordantes do que a automática. Para o e5_multilingual ($p = 0,600$) e para o bertimbau_pt ($p = 0,614$), não há evidência estatística

de que uma condição de instrução seja superior à outra: as diferenças de F1-macro reportadas na Tabela 4 para esses dois modelos podem ser atribuídas a ruído amostral. Esse achado é relevante para a pergunta de pesquisa, pois indica que a equivalência entre instrução manual e automática não é uniforme entre arquiteturas: ela se sustenta para o modelo com fine-tuning instrucional mais raso (e5_multilingual) e para o modelo de controle não instrucional (bertimbau_pt), mas não para o modelo com o treinamento instrucional mais extenso (e5_large_instruct), que é justamente aquele em que seria mais razoável esperar sensibilidade à qualidade da instrução.

4.5 Análise de Erros entre Condições e Modelos

Complementarmente às métricas agregadas, comparou-se diretamente os conjuntos de comentários em que cada modelo erra sob cada condição de instrução. A Tabela 7 mostra, para cada modelo, quantos casos são errados exclusivamente sob instrução automática e quantos são errados exclusivamente sob instrução manual, de um total de 1.050 comentários no teste.

Tabela 7 - Erros exclusivos de cada condição de instrução, por modelo (teste, n = 1.050)

Modelo	So erra c/ instrucao automatica	Só erra c/ instrução manual
e5_multilingual	62	69
e5_large_instruct	68	39
bertimbau_pt (controle)	52	46

Fonte: Elaborada pelos autores (2026)

A assimetria mais acentuada aparece novamente no e5_large_instruct: há quase o dobro de casos em que apenas a instrução automática falha (68) em comparação com casos em que apenas a manual falha (39), reforçando a diferença estatisticamente significativa reportada na Seção 4.4. Para os outros dois modelos, os erros exclusivos de cada condição são numericamente próximos (62 contra 69 no e5_multilingual; 52 contra 46 no bertimbau_pt), consistente com a ausência de significância estatística observada para eles.

Uma segunda checagem observa quantos comentários são classificados incorretamente pelos dois modelos instrucionais ao mesmo tempo (e5_multilingual e e5_large_instruct), sob cada condição, um indício de erros correlacionados entre arquiteturas que efetivamente seguem instruções. Sob instrução manual, os dois modelos erram simultaneamente em 55 dos 1.050 comentários de teste; sob instrução automática, esse número quase dobra para 92 casos. Isso sugere que a instrução gerada automaticamente pela LLM, mesmo sendo específica por instância, tende a induzir um padrão de erro mais correlacionado entre os dois modelos instrucionais do que a instrução manual fixa, possivelmente porque instruções automáticas malsucedidas (por exemplo,

genéricas demais ou focadas no aspecto errado do comentário) afetam de forma semelhante ambos os modelos que efetivamente tentam segui-la, enquanto uma única instrução manual bem calibrada tende a produzir efeitos mais estáveis e, por isso, menos correlacionados entre erros dos dois modelos.

Quando os três modelos, incluindo o bertimbau_pt de controle, são considerados em conjunto, o número de erros idênticos entre eles é de 39 sob ausência de instrução, 29 sob instrução manual e 49 sob instrução automática. Como o bertimbau_pt não segue instruções, conforme já ressaltado na Seção 3.2, sua participação nesses erros idênticos é mais um indício de que o comentário em questão é intrinsecamente difícil de classificar, provavelmente por envolver ironia, ambiguidade ou ofensas explícitas, e não necessariamente evidência de que a instrução manual ou automática o confundiu especificamente. Ainda assim, o menor número de erros conjuntos sob instrução manual (29) é compatível com a leitura de que essa condição produz o comportamento mais consistente entre arquiteturas distintas, instrucionais ou não.

4.6 Geometria do Espaço de Embeddings: Separação entre Classes

Para investigar se as instruções alteram a organização do espaço vetorial independentemente do classificador treinado sobre ele, calculou-se, para cada combinação de modelo e condição, a similaridade de cosseno média entre pares de comentários da mesma classe (intraclasse) e de classes distintas (interclasse), em uma amostra de 500 comentários do teste. A diferença entre essas duas medidas — aqui chamada de separação — indica o quanto a condição de instrução aproxima comentários semelhantes e afasta comentários de classes diferentes no espaço de embeddings. A projeção em duas dimensões via PCA dos mesmos embeddings de teste (disponível no notebook experimental) mostra visualmente uma sobreposição substancial entre as classes "ofensivo" e "não ofensivo" nas três condições, para o modelo e5_large_instruct, o que é consistente com os valores de separação relativamente baixos reportados na Tabela 8.

Tabela 8 - Similaridade de cosseno intra e interclasse e separação resultante, por modelo e condição (teste, amostra de 500 comentários)

Modelo	Condicao	Intraclasse	Interclasse	Separacao
e5_multilingual	Sem instrução	0,862	0,858	0,004
e5_multilingual	Instrução manual	0,932	0,931	0,001
e5_multilingual	Instrução automática	0,867	0,860	0,007
e5_large_instruct	Sem instrução	0,851	0,841	0,009
e5_large_instruct	Instrução manual	0,953	0,948	0,005
e5_large_instruct	Instrução automática	0,880	0,862	0,018
bertimbau_pt (controle)	Sem instrução	0,261	0,213	0,048
bertimbau_pt (controle)	Instrucao manual	0,718	0,705	0,013
bertimbau_pt (controle)	Instrução automática	0,417	0,366	0,052

Fonte: Elaborada pelos autores (2026), a partir dos resultados do notebook experimental.

Dois pontos chamam atenção nessa tabela. Primeiro, para os modelos `e5_multilingual` e `e5_large_instruct`, a instrução manual eleva tanto a similaridade intraclasse quanto a interclasse para patamares muito próximos de 0,93–0,95, reduzindo a separação relativa entre as classes — um efeito de homogeneização do espaço vetorial, no qual a instrução fixa (idêntica para todos os comentários) parece dominar a representação final e tornar os embeddings, de modo geral, mais similares entre si, independentemente da classe. Já a instrução automática, por variar de um comentário para outro, evita esse efeito de homogeneização e produz a maior separação entre as classes nos dois modelos instrucionais (0,007 e 0,018, respectivamente), superior inclusive à da condição sem instrução.

Esse resultado é aparentemente contraditório com o desempenho de classificação reportado na Seção 4.1, em que a instrução manual supera a automática para o `e5_large_instruct`. A explicação mais provável é que a métrica de separação linear simples (diferença entre médias de similaridade de cosseno) captura apenas um aspecto da estrutura do espaço vetorial, enquanto o MLP treinado sobre os embeddings pode explorar fronteiras de decisão não lineares que não dependem diretamente dessa métrica agregada. Em outras palavras, uma maior separação média entre classes no espaço de cosseno não garante, por si só, uma fronteira de decisão mais fácil de aprender por um classificador supervisionado, os dois fenômenos podem divergir, como ocorre aqui. Para o `bertimbau_pt`, que não segue instrução, o comportamento é distinto: a concatenação de qualquer texto adicional (manual ou automático) eleva substancialmente as similaridades absolutas em relação à ausência de instrução, mas a maior separação relativa ocorre justamente nas condições sem instrução e com instrução automática, e não na manual, Reforçando que, nesse modelo, o efeito observado decorre da adição de contexto textual, e não de uma compreensão semântica da instrução enquanto tal.

5. Conclusão

Retomando a pergunta central deste trabalho, se instruções geradas automaticamente por LLMs são comparáveis a instruções escritas manualmente para embeddings instrucionais na detecção de linguagem ofensiva em português brasileiro, os resultados sugerem uma resposta condicionada ao grau de treinamento instrucional do modelo de embeddings. Para o `e5_multilingual`, cujo fine-tuning instrucional é mais superficial, e para o `bertimbau_pt`, que não segue instruções, a diferença entre instrução manual e automática não é estatisticamente significativa (Seção 4.4): nesses casos, a instrução gerada automaticamente pode ser considerada uma alternativa viável à escrita manual, sem perda de desempenho comprovada. Já para o `e5_large_instruct`, o modelo com

o treinamento instrucional mais extenso e, portanto, aquele em que a hipótese de comparabilidade seria mais interessante de confirmar, a instrução manual supera a automática de forma estatística e consistentemente robusta a variações de semente (Seções 4.3 e 4.4), o que não sustenta a hipótese de equivalência nesse caso específico.

Um achado adicional, não antecipado na formulação original da pergunta de pesquisa, é que, para os dois modelos instrucionais, a condição sem qualquer instrução obteve desempenho igual ou superior às duas condições instruídas (Seção 4.1). Isso indica que, ao menos neste pipeline — embeddings pré-treinados seguidos de um classificador MLP treinado especificamente para a tarefa —, a adição de instrução textual não trouxe ganho de F1-macro em relação ao uso direto do texto bruto, contrariando a intuição inicial de que instruções deveriam ajudar por si só. O benefício de uma instrução bem escrita parece residir mais na disputa entre instrução manual e automática do que na comparação entre instrução e ausência de instrução, o que sugere que trabalhos futuros nessa linha talvez devam concentrar a pergunta de pesquisa nessa comparação mais restrita, em vez de tratar "ter instrução" como necessariamente vantajoso.

Por fim, independentemente da condição de instrução, todas as combinações de embeddings avaliadas superaram com folga os baselines externos de TF-IDF + Regressão Logística e Heurística Lexical (Seção 4.2), confirmando o valor de representações vetoriais densas para a tarefa. Como limitações deste estudo, destacam-se: o uso de um único modelo de linguagem (llama-3.1-8b-instant, via API do Groq) para a geração automática de instruções, sem validação humana da qualidade dessas instruções; uma única arquitetura de classificador (MLP com camadas 256 e 128); e um único conjunto de dados, de domínio político e coletado no Instagram, o que limita a generalização das conclusões para outros domínios de discurso de ódio ou para outras combinações de modelo de embeddings e classificador.

Referências

ASAI, Akari et al. **Task-aware retrieval with instructions**. [S. l.: s. n.], 2022. Disponível em: <https://arxiv.org/abs/2211.09260>. Acesso em: 26 jul. 2026.

GHORBANPOUR, Faeze; DEMENTIEVA, Daryna; FRASER, Alexander. **Can prompting LLMs unlock hate speech detection across languages? A zero-shot and few-shot study**. [S. l.: s. n.], 2025. Disponível em: <https://aclanthology.org/2025.woah-1.39.pdf>. Acesso em: 26 jun. 2026.

OLIVEIRA, Amanda et al. **How good is ChatGPT for detecting hate speech in Portuguese?** [S. l.: s. n.], 2023. Disponível em: <https://aclanthology.org/2023.stil-1.10.pdf>. Acesso em: 26 jun. 2026.

OLIVEIRA, Amanda et al. **Toxic speech detection in Portuguese: a comparative study of large language models**. [S. l.: s. n.], 2024a. Disponível em: <https://aclanthology.org/2024.propor-1.11.pdf>. Acesso em: 26 jun. 2026.

OLIVEIRA, Amanda et al. **Toxic text classification in Portuguese: is LLaMA 3.1 8B all you need?** [S. l.: s. n.], 2024b. Disponível em: <https://aclanthology.org/2024.stil-1.15.pdf>. Acesso em: 26 jun. 2026.

SU, Hongjin et al. **One embedder, any task: instruction-finetuned text embeddings**. [S. l.: s. n.], 2022. Disponível em: <https://arxiv.org/abs/2212.09741>. Acesso em: 26 jul. 2026.

VARGAS, Francielle; CARVALHO, Isabelle. **franciellevargas/HateBR: v7.0.0**. Zenodo, 2025. DOI: 10.5281/zenodo.15651261. Disponível em: <https://doi.org/10.5281/zenodo.15651261>. Acesso em: 16 jul. 2026.

WANG, Liang et al. **Multilingual E5 text embeddings: a technical report**. [S. l.: s. n.], 2024. Disponível em: <https://arxiv.org/abs/2402.05672>. Acesso em: 26 jul. 2026.

YUAN, Y. *et al.* Query Understanding in LLM-based Conversational Information Seeking. **Companion Proceedings of the ACM on Web Conference 2025**, New York, NY, USA, p. 73–76, 2025. Disponível em: <https://arxiv.org/html/2504.06356v1> . Acesso em: 28 jul. 2026.

ZHANG, Yuan; BERTAGLIA, Thales. **LinGO: a linguistic graph optimization framework with LLMs for interpreting intents of online uncivil discourse**. [S. l.: s. n.], 2025. Disponível em: <https://arxiv.org/abs/2602.04693>. Acesso em: 26 jun. 2026.

ZHANG, Zhihan et al. **Auto-Instruct: automatic instruction generation and ranking for black-box language models**. [S. l.: s. n.], 2023. Disponível em: <https://arxiv.org/abs/2310.13127>. Acesso em: 26 jul. 2026.

ZHOU, Yongchao et al. **Large language models are human-level prompt engineers**. [S. l.: s. n.], 2022. Disponível em: <https://arxiv.org/abs/2211.01910>. Acesso em: 26 jul. 2026.